A függvény meghatározása nagyon egyszerű: ez egy kis kódrészlet, ami valami specifikus feladatot lát el. A függvény az utasítások olyan blokkja, ami végrehajtja a benne meghatározott művelet(ek)et, és miután lefutott, tetszőlegesen újra hívható, futtatható. A függvények modulárisabbá, áttekinthetőbbé teszik a kódot, lehetővé téve ugyanazon kódrészlet (függvény) újra és újra használatát.

A Python számos beépített függvénnyel rendelkezik, amikkel már megismerkedtünk, többek között:

- print (), ami adatokat nyomtat a terminálra
- int (), ami egy karakterlánc vagy szám adattípust egész adattípussá konvertál
- len (), ami egy objektum (pl. string) hosszát adja vissza

A függvénynevek zárójeleket tartalmaznak, és tartalmazhatnak paramétereket.

Ebben a leckében azt vizsgáljuk, meg hogy, hogyan készíthetünk saját függvényeket, amiket a kódolási projektjeinkben használhatunk.

A FÜGGVÉNYEK LÉTREHOZÁSA A DEF KULCSSZÓVAL TÖRTÉNIK, A KÖVETKEZŐ MÓDON:

```
1. def fuggveny(param1):
2. """Ez egy függvény"""
3. # ... jelenleg nem sokat csinál
4. print "semmi"
5. return None
```

A def kulcsszót a függvény neve követi, majd utána zárójelben a függvénynek átadandó paramétereket soroljuk fel, jelen esetben ez csak egy, param1 nevű paramétert jelent. Ezután megadhatunk a függvényben egy ún. doc-stringet, amiben leírhatjuk pár szóval, hogy mit csinál, majd megírjuk a függvény törzsét. A függvény visszatérésekor a példánkban a „None” értéket adja.
Első példánk legyen egy klasszikus „Hello, World” függvény! Az első lépés a függvény létrehozása:

```
1. def hello():
```

Egyelőre a függvény neve után álló zárójelek üresek. A függvény létrehozása után következik az a kódrészlet, ami majd a függvény hívásakor lefut:

```
def hello():
    print("Hello, World!")
```

A függvényünk elkészült, ha ezek után beillesztjük egy programba, akkor még semmit nem fog csinálni. Ahhoz, hogy működjön, meg kell hívnunk, ami a nevével történik:

```
def hello():
  print("Hello, World!")

hello()
```

Ez a program még tényleg nem csinál sok hasznosat, a következővel azonban ki tudjuk számoltatni pl. egy háromszög kerületét:

```
def triangle(a,b,c):
    print("perimeter: ",a+b+c)

triangle(3,4,5)
```

Sokkal rugalmasabb a program, ha az oldalhosszakat mi adhatjuk meg!

```
def triangle(a,b,c):
  print("ker: ",a+b+c)

a = input("a oldal: ")
b = input("b oldal: ")
c = input("c oldal: ") 7.

triangle(a,b,c)
```

A meglepő eredményt a hibás változó típus alkalmazása okozza, a program string-ként dolgozza fel az értékeket. A program helyesen:

```
def triangle(a,b,c):
  print("ker: ",a+b+c)
```

```
a = float(input("a oldal: "))
b = float(input("b oldal: "))
c = float(input("c oldal: ")) 7.

triangle(a,b,c)
```

A „RETURN” KUKLCSSZÓ ÉS A VISSZATÉRÉSI ÉRTÉK

A „return” kulcsszót arra használhatjuk, hogy segítségével kilépjünk a függvényből, és a főprogram azon pontjára kerüljünk vissza, ahol a hívás történt. A „return” tartalmazhat visszatérési értéket is, ami például egy számítás eredménye lehet.
A következő példa egy olyan függvény, ami a megadott szám abszolút értékét számolja ki, majd visszatérve a hívás helyére az eredményt megjeleníti a konzolon:

```
def absolute_value(num):
  if num >= 0:
  return num
  else:
  return -num



num = float(input("enter a number: "))
print(absolute_value(num))
```

Egy másik példa két szám összegzését végzi:

```
def add_numbers(x,y):
  sum = x + y
  return sum

num1 = float(input("num1: "))
num2 = float(input("num2: ")) 7.

print("The sum is", add_numbers(num1, num2))
```